

Documentación: El Hook `useEffect` en React (con JavaScript)

¿Para qué se usa `useEffect`?

El Hook `useEffect` te permite realizar **efectos secundarios** (o *side effects*) en tus componentes funcionales de React. Un efecto secundario es cualquier cosa que interactúa con el mundo exterior de tu componente o que no es puramente el cálculo del *renderizado* (la salida JSX).

Ejemplos comunes de efectos secundarios:

1. **Manipulación del DOM:** Cambiar el título del documento (`document.title`).
2. **Llamadas a API:** Obtener datos de un servidor externo.
3. **Suscripciones:** Configurar o limpiar escuchadores de eventos, temporizadores (`setTimeout`, `setInterval`).
4. **Sincronización:** Interactuar con librerías externas que no son de React.

En resumen, `useEffect` es la forma en que React maneja lo que en los componentes de clase se hacía con los métodos del ciclo de vida (`componentDidMount`, `componentDidUpdate` y `componentWillUnmount`).

¿Cómo se usa `useEffect`?

El Hook `useEffect` acepta dos argumentos:

1. **Una función de efecto:** La función que contiene el código del efecto secundario.
2. **Un Array de dependencias (opcional):** Un *array* de valores que le dice a React cuándo debe volver a ejecutar el efecto.

Sintaxis básica

```
JavaScript
import React, { useState, useEffect } from 'react';

function MiComponente() {
  // El primer argumento es la función de efecto
  // El segundo argumento es el array de dependencias (opcional)
  useEffect(() => {
    // Código del efecto secundario
    console.log("¡El efecto se ha ejecutado!");
  }, [/* Array de dependencias */]);

  // ... (resto del componente)
}
```

□ Ejemplos Explicados

Ejemplo 1: Ejecutar una vez al montar (Simulando `componentDidMount`)

Si pasas un *array* de dependencias **vacío** (`[]`), React ejecutará el efecto **solo una vez** después del *renderizado* inicial del componente.

□ Caso de uso: Cargar datos de una API.

JavaScript

```
import React, { useState, useEffect } from 'react';

function ComponenteDeDatos() {
  const [datos, setDatos] = useState(null);
  const [cargando, setCargando] = useState(true);

  // El array de dependencias vacío [] asegura que esto solo se
  // ejecute al MONTARSE
  useEffect(() => {
    // 1. Iniciar la carga de datos
    console.log('Componente montado. Realizando llamada a la API...');

    // Simulamos una llamada a la API
    fetch('https://jsonplaceholder.typicode.com/posts/1')
      .then(response => response.json())
      .then(data => {
        setDatos(data);
        setCargando(false);
      });
  }, []); // <-- Array de dependencias vacío

  if (cargando) {
    return <p>Cargando datos...</p>;
  }

  return (
    <div>
      <h3>Datos Cargados</h3>
      <p>Título: {datos.title}</p>
    </div>
  );
}
```

Explicación: Al estar `[]` vacío, React sabe que el efecto no depende de ninguna *prop* o *estado* dentro del componente, por lo que solo necesita ejecutarlo una vez, al igual que `componentDidMount` en componentes de clase.

Ejemplo 2: Ejecutar en cada *renderizado* (Sin Array de Dependencias)

Si **omites por completo** el segundo argumento (*array* de dependencias), el efecto se ejecutará **después de cada *renderizado*** del componente.

□ Caso de uso: Sincronizar el título de la página con el estado.

JavaScript

```
import React, { useState, useEffect } from 'react';

function ComponenteContador() {
  const [contador, setContador] = useState(0);

  // No hay array de dependencias: se ejecuta después de CADA
  renderizado
  useEffect(() => {
    document.title = `Has pulsado ${contador} veces`;
    console.log('El título de la página se ha actualizado.');
```

}); // <-- Sin array de dependencias

```

  return (
    <div>
      <p>Contador: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>
        Incrementar
      </button>
    </div>
  );
}
```

Explicación: Cada vez que haces clic en el botón, el estado `contador` cambia, el componente se *re-renderiza*, y `useEffect` se vuelve a ejecutar, actualizando el título de la página para reflejar el nuevo valor.

Ejemplo 3: Ejecutar solo cuando las dependencias cambian (Simulando `componentDidUpdate`)

Si incluyes variables en el *array* de dependencias, el efecto se ejecutará solo en el montaje y cuando **cualquiera de esas variables cambie**.

❑ **Caso de uso: Recargar datos basados en una *prop* o *estado*.**

JavaScript

```
import React, { useState, useEffect } from 'react';

function ComponentePerfil({ userId }) {
  const [perfil, setPerfil] = useState(null);

  // El efecto se ejecuta cuando userId (la dependencia) cambia.
  useEffect(() => {
    console.log(`Cargando perfil para el ID: ${userId}`);
    // Simulamos la carga del perfil
    fetch(`https://api.ejemplo.com/usuarios/${userId}`)
      .then(res => res.json())
      .then(data => setPerfil(data));

    // Cleanup: Opcional, pero se recomienda.
    return () => {
      console.log(`Limpieza: Cancelando carga de perfil anterior
        (${userId})`);
    };
  }, [userId]); // <-- Dependencia en userId

  return (
    <div>
```

```

    <h3>Perfil del Usuario ID: {userId}</h3>
    {perfil ? <p>Nombre: {perfil.nombre}</p> : <p>Cargando...</p>}
  </div>
);
}

```

Explicación: React compara el valor de `userId` entre *renderizados*. Si es diferente, ejecuta el efecto de nuevo. Si es el mismo (por ejemplo, otras *props* o *estados* han cambiado, pero `userId` no), **el efecto se omite**.

□ La Función de Limpieza (Cleanup)

Para evitar fugas de memoria y manejar suscripciones correctamente, `useEffect` puede devolver opcionalmente una función de **limpieza** (*cleanup function*).

Esta función se ejecuta en dos momentos clave:

1. **Antes de que se ejecute la próxima vez:** Se ejecuta antes de cada nueva ejecución del efecto (si las dependencias han cambiado).
2. **Cuando el componente se desmonta:** Se ejecuta justo antes de que el componente sea eliminado del DOM (simulando `componentWillUnmount`).

□ Caso de uso: Temporizadores (`setInterval`).

JavaScript

```

import React, { useState, useEffect } from 'react';

function ComponenteTemporizador() {
  const [segundos, setSegundos] = useState(0);

  useEffect(() => {
    // Configuración del efecto (iniciar el temporizador)
    const intervaloId = setInterval(() => {
      setSegundos(prevSeg => prevSeg + 1);
    }, 1000);

    // FUNCIÓN DE LIMPIEZA
    return () => {
      // Limpiamos el intervalo cuando el componente se desmonte
      clearInterval(intervaloId);
      console.log('¡Temporizador limpiado!');
    };
  }, []); // Solo se ejecuta en el montaje y la limpieza en el
  desmontaje

  return <h2>Tiempo transcurrido: {segundos} segundos</h2>;
}

```

Explicación: Si no limpias el intervalo (con `clearInterval`) al desmontar el componente, el temporizador seguiría ejecutándose en segundo plano, intentando actualizar un estado de un componente que ya no existe. La función de limpieza evita este problema.

❓ Ejemplo Avanzado: `useEffect` con `async/await` (Llamada a API)

Aunque la función que le pasas directamente a `useEffect` **no puede ser `async`**, puedes definir una función asíncrona **dentro** de ella y luego llamarla.

El Problema

Si intentas definir la función de efecto como asíncrona, React te dará un error:

JavaScript

```
// ❌ ¡ESTO NO FUNCIONARÁ DIRECTAMENTE!  
useEffect(async () => {  
  const respuesta = await fetch('...');  
  // ...  
}, []);
```

✅ La Solución Recomendada: Función Anidada

La mejor práctica es definir una función asíncrona separada dentro del cuerpo de `useEffect` y llamarla inmediatamente.

Componente de Ejemplo: Carga de Lista de Usuarios

Este ejemplo muestra cómo cargar una lista de usuarios de una API externa utilizando `async/await` y cómo manejar el estado de carga y posibles errores.

JavaScript

```
import React, { useState, useEffect } from 'react';  
  
function ListaDeUsuarios() {  
  const [usuarios, setUsuarios] = useState([]);  
  const [cargando, setCargando] = useState(true);  
  const [error, setError] = useState(null);  
  
  // 1. El useEffect se ejecuta una sola vez al montar  
  useEffect(() => {  
  
    // 2. Definimos una función anidada que SÍ es asíncrona  
    const obtenerUsuarios = async () => {  
      try {  
        // Reiniciamos estados por si se ejecuta de nuevo (aunque aquí  
        // es solo una vez)  
        setCargando(true);  
        setError(null);  
  
        // 3. Usamos await para esperar la respuesta de la red  
        const respuesta = await  
        fetch('https://jsonplaceholder.typicode.com/users');  

```

```

    // Manejo de errores HTTP (por ejemplo, 404 Not Found)
    if (!respuesta.ok) {
      throw new Error(`Error HTTP: ${respuesta.status}`);
    }

    // 4. Usamos await para esperar el parsing del JSON
    const datos = await respuesta.json();

    // 5. Actualizamos el estado con los datos
    setUsuarios(datos);

  } catch (err) {
    // 6. Manejo de errores de red o del código
    console.error("Hubo un error al obtener los usuarios:", err);
    setError(err.message);
  } finally {
    // 7. Siempre finalizamos el estado de carga
    setCargando(false);
  }
};

// 8. Invocamos la función asíncrona inmediatamente
obtenerUsuarios();

}, []); // Array de dependencias vacío: se ejecuta solo en el
montaje

// --- Lógica de Renderizado ---
if (cargando) {
  return <p>⬜ Cargando lista de usuarios...</p>;
}

if (error) {
  return <p style={{ color: 'red' }}>⬜ Error al cargar los datos:
{error}</p>;
}

return (
  <div>
    <h3>⬜ Lista de Usuarios Cargada</h3>
    <ul>
      {usuarios.map(usuario => (
        <li key={usuario.id}>{usuario.name} ({usuario.email})</li>
      ))}
    </ul>
  </div>
);
}

```

□ Explicación del Patrón `async/await` en `useEffect`

1. **Definición de la Función Asíncrona:** Dentro de `useEffect`, definimos una función regular (en este caso, `obtenerUsuarios`) y le añadimos la palabra clave `async`. Esto la convierte en una función que devuelve una `Promise` y nos permite usar `await` en su interior.
2. **Manejo de Promesas con `await`:** Utilizamos `await` para pausar la ejecución del código de forma **no bloqueante** hasta que la `Promise` de `fetch` se resuelva.

Hacemos esto dos veces: una para la respuesta de la red, y otra para el *parsing* del cuerpo a JSON.

3. **Bloque `try...catch`:** Es **fundamental** envolver el código `async/await` en un bloque `try...catch`. Esto nos permite capturar cualquier error (de red, de la API, o de la promesa) de manera controlada y actualizar el estado de error (`setError`), lo cual es clave para la experiencia del usuario.
4. **Invocación Inmediata:** Llamamos a la función `obtenerUsuarios()` inmediatamente después de definirla. Esto inicia el proceso asíncrono tan pronto como `useEffect` se ejecuta.
5. **Control del Estado de Carga:** Los estados `cargando` y `error` nos permiten mostrar mensajes apropiados al usuario en diferentes momentos del ciclo de vida de la solicitud.

Este patrón es el estándar para realizar operaciones asíncronas de manera limpia y controlada dentro de los *Hooks* de React.

Ejemplo 4: `useEffect` con Event Listeners y Limpieza

Cuando trabajas con eventos globales (`window.addEventListener`), es fundamental que, al desmontarse el componente, elimines el *listener* correspondiente (`window.removeEventListener`). Si no lo haces, el *listener* seguirá activo e intentará ejecutar código en un componente que ya no existe, causando fugas de memoria y errores.

Caso de uso: Escuchar la tecla "Escape" para cerrar un modal.

JavaScript

```
import React, { useState, useEffect } from 'react';

function ComponenteConModal() {
  const [modalAbierto, setModalAbierto] = useState(false);

  // El efecto se encarga de escuchar el evento de teclado
  useEffect(() => {

    // 1. Definir el manejador de eventos
    const handleEscKey = (evento) => {
      // Solo si la tecla pulsada es 'Escape' y el modal está abierto,
      lo cerramos
      if (evento.key === 'Escape' && modalAbierto) {
        setModalAbierto(false);
        console.log("Modal cerrado por tecla 'Escape'");
      }
    };

    // 2. Adjuntar el listener al objeto global 'window'
    window.addEventListener('keydown', handleEscKey);
    console.log("Listener de teclado 'keydown' añadido.");

    // 3. FUNCIÓN DE LIMPIEZA
    // Esta función se ejecuta cuando el componente se desmonta.
    return () => {
```

```

        // 4. Eliminar el listener (es CRUCIAL para evitar fugas de
        memoria)
        window.removeEventListener('keydown', handleEscKey);
        console.log("Listener de teclado 'keydown' ELIMINADO.");
    };

    /* * DEPENDENCIAS:
    * El efecto debe ejecutarse de nuevo si 'modalAbierto' cambia,
    * para que la función handleEscKey tenga siempre el valor más
    reciente.
    */
    }, [modalAbierto]);

    // Si no se incluyera modalAbierto en las dependencias, la función
    // handleEscKey usaría el valor inicial de modalAbierto (false),
    // y nunca podría cerrarse si se abriera después.

    return (
        <div>
            <button onClick={() => setModalAbierto(true)}
            disabled={modalAbierto}>
                {modalAbierto ? 'Modal Abierto' : 'Abrir Modal (Pulsa ESC para
                cerrar)'}
            </button>

            {modalAbierto && (
                <div style={modalStyle}>
                    <h4>Modal Activo</h4>
                    <p>Pulsa la tecla ESC o el botón para cerrar.</p>
                    <button onClick={() =>
                    setModalAbierto(false)}>Cerrar</button>
                </div>
            )}
        </div>
    );
}

// Estilos simples para el modal (solo para el ejemplo)
const modalStyle = {
    border: '2px solid #333',
    padding: '20px',
    margin: '15px',
    backgroundColor: '#f9f9f9',
};

// Exportamos el componente para que pueda ser usado
// export default ComponenteConModal;

```

□ ¿Por qué es Vital la Limpieza aquí?

1. **Montaje:** Cuando el `ComponenteConModal` se muestra por primera vez, el `useEffect` se ejecuta, adjuntando la función `handleEscKey` a `window`.
2. **Desmontaje:** Si navegas a otra página o el componente deja de renderizarse, la función de limpieza (`return () => {...}`) se ejecuta. Esta línea:

JavaScript

```
window.removeEventListener('keydown', handleEscKey);
```


desconecta el código de tu componente del navegador, asegurando que no haya efectos secundarios ejecutándose innecesariamente.

3. **Dependencias:** Incluir `modalAbierto` en el *array* de dependencias (`[modalAbierto]`) es esencial. Si este valor cambia (por ejemplo, el modal se abre), React re-ejecuta el `useEffect`. Esto garantiza que la nueva función `handleEscKey` creada dentro del efecto **conozca el valor más reciente** de `modalAbierto` (el que está "fresco" en el *closure* de React).
-